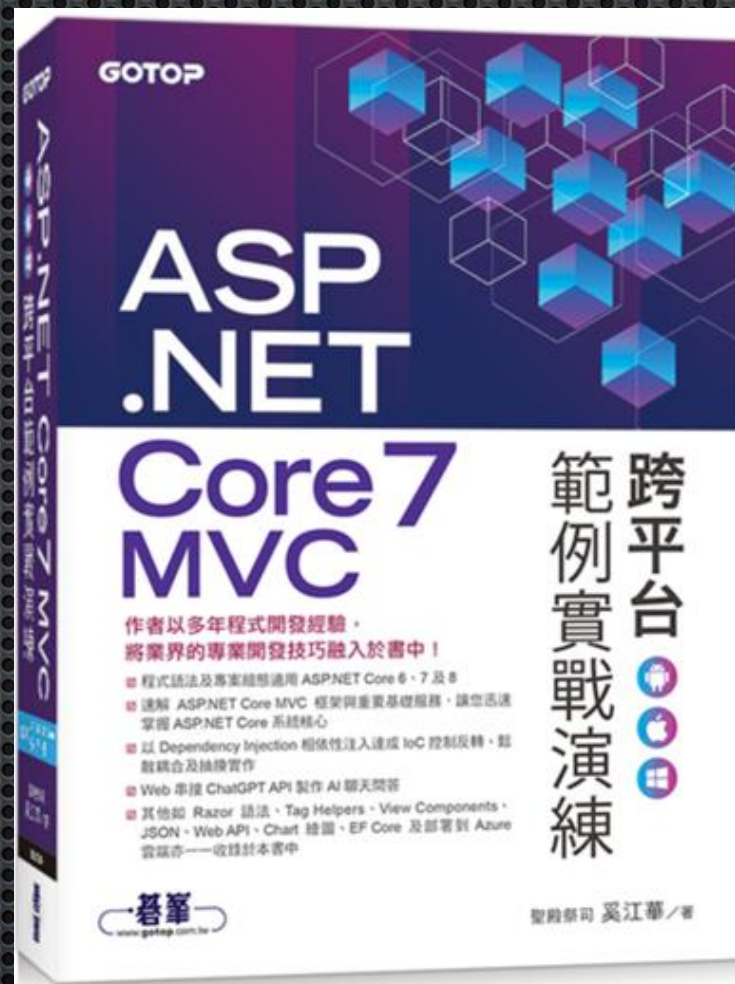


第04章

ASP.NET Core 7 框架與基礎服務



作者：聖殿祭司。奚江華

大綱(1)

4-1 ASP.NET Core 框架簡介

4-2 ASP.NET Core Fundamentals 基礎服務概觀

4-3 重要基礎服務簡介

4-3-1 ASP.NET Core 應用程式載入過程

4-3-2 本機開發電腦環境組態檔–launchSettings.json

4-3-3 Program.cs–Main()建立 Host 主機

4-3-4 DI相依性注入與 Middleware 中介元件

4-3-5 Configuration 組態

大綱(2)

4-3-6 Options Pattern 選項模式

4-3-7 Environment 環境

4-3-8 Content Root 與 Web Root

4-3-9 Logging 記錄

ASP.NET Core 框架簡介

4-1

ASP.NET Core 框架與服務

- ASP.NET Core 是一個跨平台、高效能、開源框架，用來建立現代化、雲端基礎、Internet 連結（指 IoT）的應用程式
- 背後支撐整個框架運作的是眾多基礎服務，包括：
Hosting、Configuration 組態系統、相依性注入、Middleware、Routing、Environment、Logging 等服務

使用 ASP.NET Core 之利益

ASP.NET Core 架構上更精簡與模組化，優點有：

- 能在 Windows、macOS 和 Linux 上開發、建置與執行
- 內建 Dependency Injection 相依性注入
- 一個輕量級、高效能和模組化 HTTP Request Pipeline
- 能夠裝載到 Kestrel、IIS、HTTP.sys、Nginx、Apache、Docker，或在你自己的程序中自我裝載（self-host）
- 支援 .NET (Core) Runtime 多版本並行

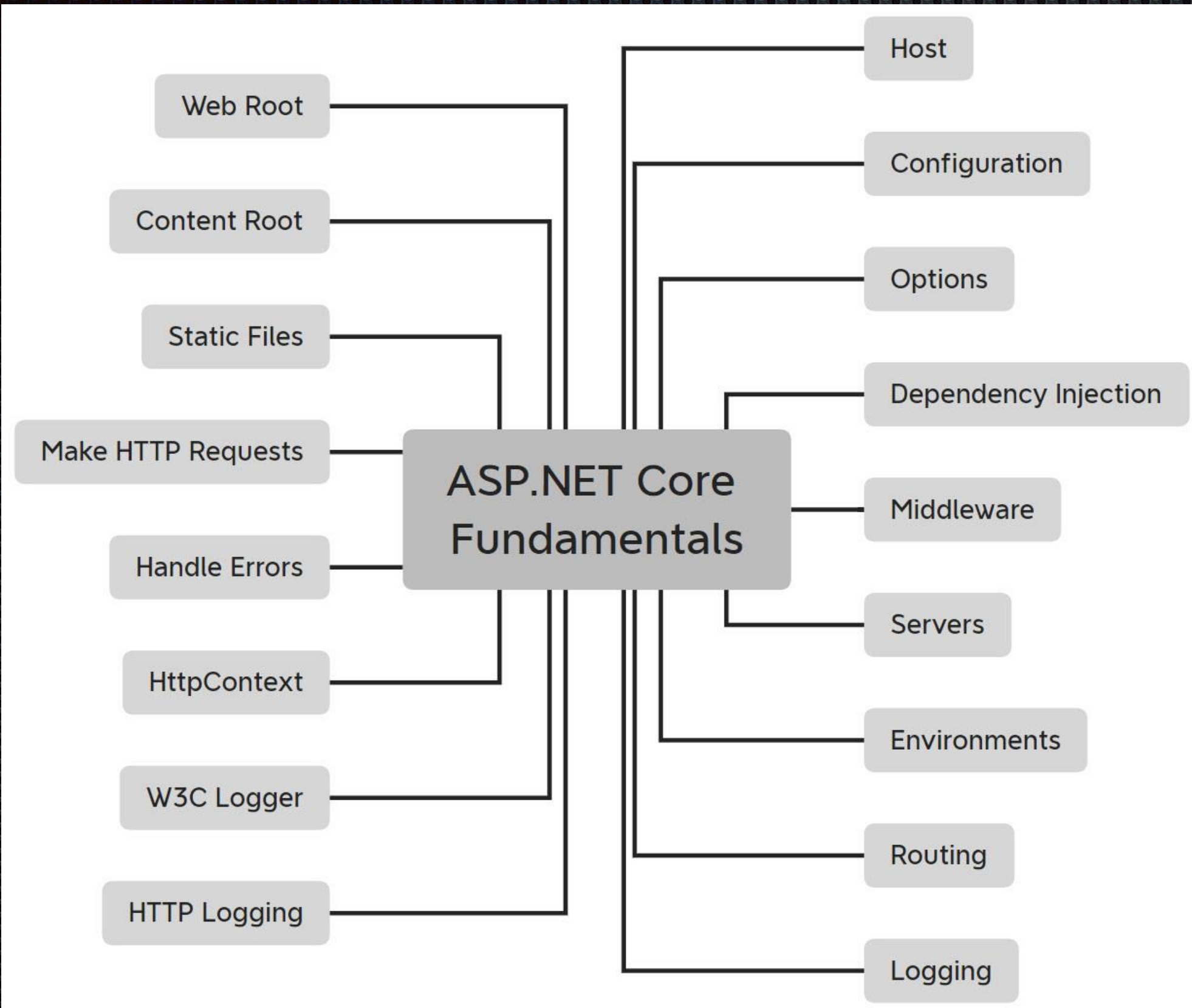
使用 ASP.NET Core 建立 Web UI 和 Web APIs

- MVC Pattern 能助 Web UI 和 Web APIs 更具有可測試性
- Razor Markup 為 MVC Views 提供更具生產力語法
- Tag Helpers 能夠讓 Server 端程式參與 Razor 檔中 HTML elements 元素的建立與轉譯
- 內建多種資料格式和內容協商，使你的 Web APIs 可以覆蓋廣泛的用戶端，包括瀏覽器和行動裝置
- Model Binding 自動將 HTTP Requests 對應到 Action 方法的參數
- Model Validation 自動執行 Client 端與 Server 端的驗證

ASP.NET Core Fundamentals 基礎服務概觀

4-2

Fundamentals 基礎服務



基礎服務功能(1)

- **Host**：裝載與執行.NET Core 應用程式的主機環境，它封裝了所有App 資源，如 Server、Middleware、DI 和 Configuration，並實作IHostedService
- **Server**：指 HTTP Server 或 Web Server 伺服器，用於監聽 HTTP 請求與回應的網頁伺服器
- **Dependency Injection**：相依性注入，亦稱 DI Container / Service Container / IoC Container
- **Middleware**：在處理 HTTP 請求的管線中，包含一系列 Middleware中介軟體元件

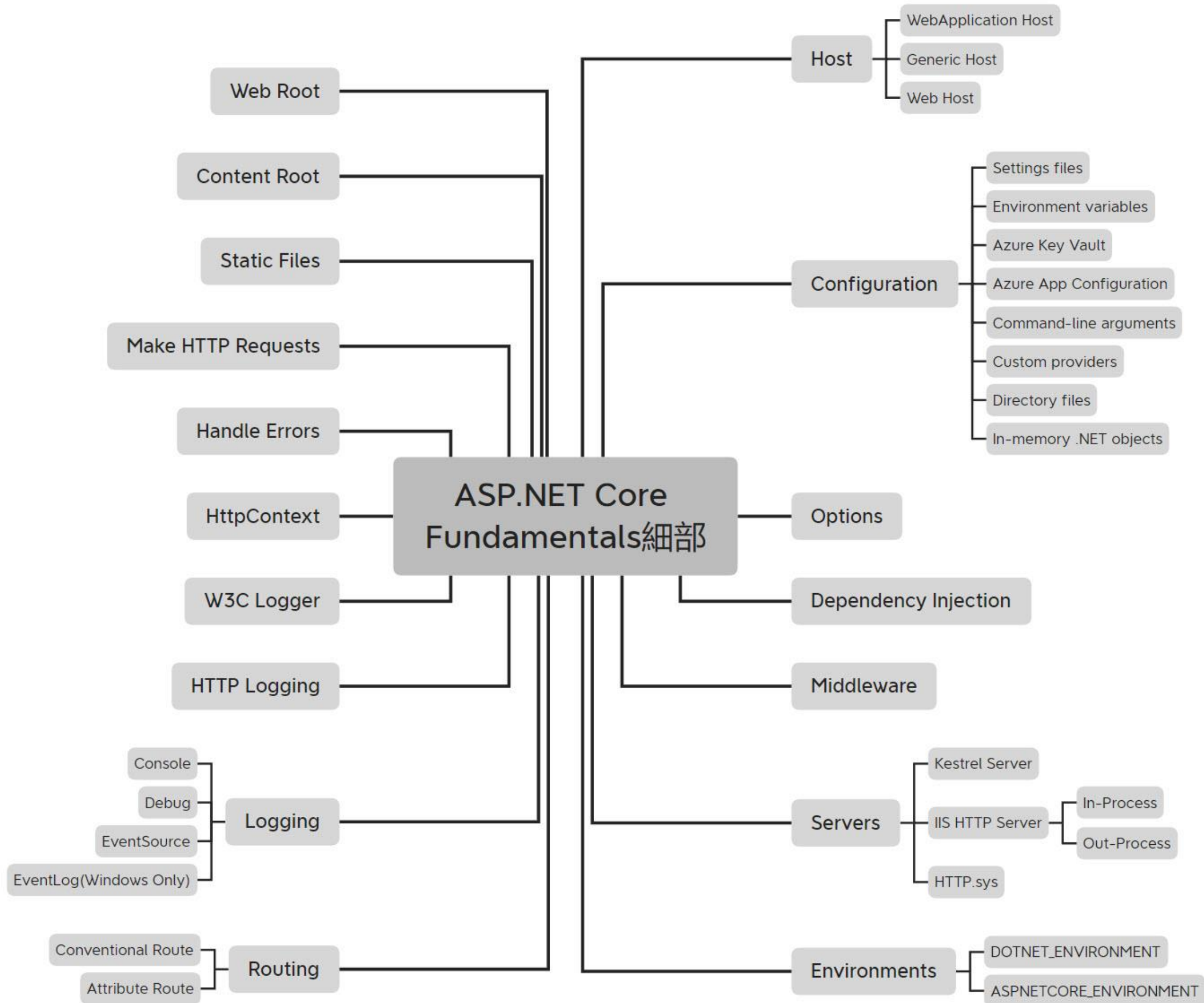
基礎服務功能(2)

- **Configuration 組態**：ASP.NET Core 的組態框架，提供 Host 和 App 所需的組態存取系統
- **Options**：是指 Options Pattern 選項模式，用類別來表示一組設定，.NET Core 中大量使用選項模式設定組態
- **Environment**：環境變數與機制，內建 Development、Staging 與 Production 三種環境
- **Logging**：資訊或事件的記錄機制

基礎服務功能(3)

- **Routing**：自 ASP.NET Core 3.0 開始採用端點路由，它負責匹配與派送HTTP 請求到應用程式執行端點
- **Handling Error**：負責錯誤處理的機制
- **Make HTTP Request**：是 IHttpConnectionFactory 實作，用於建立 HttpClient實例
- **Content Root**：內容根目錄，代表專案目前所在的基底路徑
- **Web Root**：Web 根目錄，專案對外公開靜態資產的目錄

基礎服務明細



Fundamentals 服務如何影響 ASP.NET Core App ?

- 掌控 ASP.NET Core App 系統運作
- 提供 Hosting 和 Web Server 組態設定
- 提供各種環境變數與組態值設定
- 提供多重環境組態設定：Development、Staging 和 Production
- 提供 DI 及 Middleware 設定
- 提供路由設定
- 提供效能調校、Logging 等一堆功能

重要基礎服務簡介

4-3

ASP.NET Core 應用程式載入過程

4-3-1

ASP.NET Core執行載入過程

1. dotnet run

應用程式執行

2. launchSettings.json

載入本機電腦環境組態

3. Program.cs – Main()

建立WebApplication Host開始

4. WebApplicationBuilder
初始化

載入appsettings.json等組態/
註冊DI服務

5. WebApplication Host
建立

設定Middleware中介軟體
及路由

5. Host執行

以Kestrel監聽請求/回應

執行步驟說明(1)

- ① .NET 應用程式啟動，無論是用 F5 / Ctrl + F5 / dotnet run 方式執行
- ② 首先載入 launchSettings.json 組態，此組態是供本機電腦環境使用
- ③ 執行 Program.cs，其 Main() 是程式進入點，透過 WebApplicationBuilder 建立 WebApplication Host 主機

執行步驟說明(2)

- ④ WebApplicationBuilder 初始化時，會載入環境變數與組態，以及約250 種以上框架提供的服務亦會加入到 DI Container，自訂服務亦是在 DI Container 註冊
- ⑤ WebApplicationBuilder.Build()方法建立 WebApplication Host 主機，接著設定 HTTP Request 請求的 Middleware 中介軟體與路由
- ⑥ 設定好 Host 主機所有組態和軟體服務後，呼叫 Run() 方法執行應用程式， Kestrel Web Server 開始傾聽 HTTP 請求，並回應結果

了解執行過程目的

- 可讓你串起整個基礎框架服務的執行順序，理解它們是在什麼階段被載入執行，又扮演何種角色，及彼此的關聯性
- 避免誤解基礎服務是一群零散、各自為政的服務
- 撰寫程式時，能更清楚什麼功能要在哪調整

本機開發電腦環境組態檔 launchSettings.json

4-3-2

launchSettings.json

- launchSettings.json 是本機開發電腦的環境組態檔，裡面分兩大類、三個區塊，第一類是 IIS 設定，第二類是 Profiles 設定
- launchSettings.json 僅供本機電腦使用，而不用於部署

launchSettings.json

```
{  
  // ❶ IIS 設定  
  "iisSettings": {  
    "windowsAuthentication": false,  
    "anonymousAuthentication": true,  
    "iisExpress": {  
      "applicationUrl": "http://localhost:21358|",  
      "sslPort": 44310  
    }  
  },  
  // ❷ profiles 設定  
  "profiles": {  
    "http": {  
      "commandName": "Project",  
      "dotnetRunMessages": true,  
      "launchBrowser": true,  
      "applicationUrl": "http://localhost:5239",  
      "environmentVariables": {  
        "ASPNETCORE_ENVIRONMENT": "Development"  
      }  
    },  
    "https": {  
      "commandName": "Project",  
      "dotnetRunMessages": true,  
      "launchBrowser": true,  
      "applicationUrl": "https://localhost:7299;http://localhost:5239",  
      "environmentVariables": {  
        "ASPNETCORE_ENVIRONMENT": "Development"  
      }  
    }  
  }  
}
```

Diagram annotations:

- ❶ IIS 設定 points to the `"iisSettings"` block.
- ❷ profiles 設定 points to the `"profiles"` block.
- HTTP Port 號碼 points to the `21358` in `"applicationUrl": "http://localhost:21358|"`.
- HTTPS Port 號碼 points to the `44310` in `"sslPort": 44310`.
- 啟動瀏覽器 points to the `launchBrowser": true` in the `"http"` profile.
- 應用程式 URL 網址 points to the `"http://localhost:5239"` in the `"http"` profile.
- Development 環境 points to the `"Development"` in the `"ASPNETCORE_ENVIRONMENT"` of the `"http"` profile.

Program.cs–Main() 建立 Host 主機

4-3-3

Program.cs

- Program 的 Main()主要任務是建立 Host 主機 / 執行
- 檔案的形式有傳統及最上層陳述式
- 兩種語法形式差異，僅在於是否使用 Main()方法表達程式進入點

Program.cs傳統語法

```
public class Program
{
    public static void Main(string[] args)
    {
        var builder = WebApplication.CreateBuilder(args);

        // Add services to the container.
        builder.Services.AddControllersWithViews();

        var app = builder.Build();

        // Configure the HTTP request pipeline.
        if (!app.Environment.IsDevelopment())
        {
            ...
        }

        app.MapControllerRoute(
            name: "default",
            pattern: "{controller=Home}/{action=Index}/{id?}");

        app.Run();
    }
}
```


Top Level Statement

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");

    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

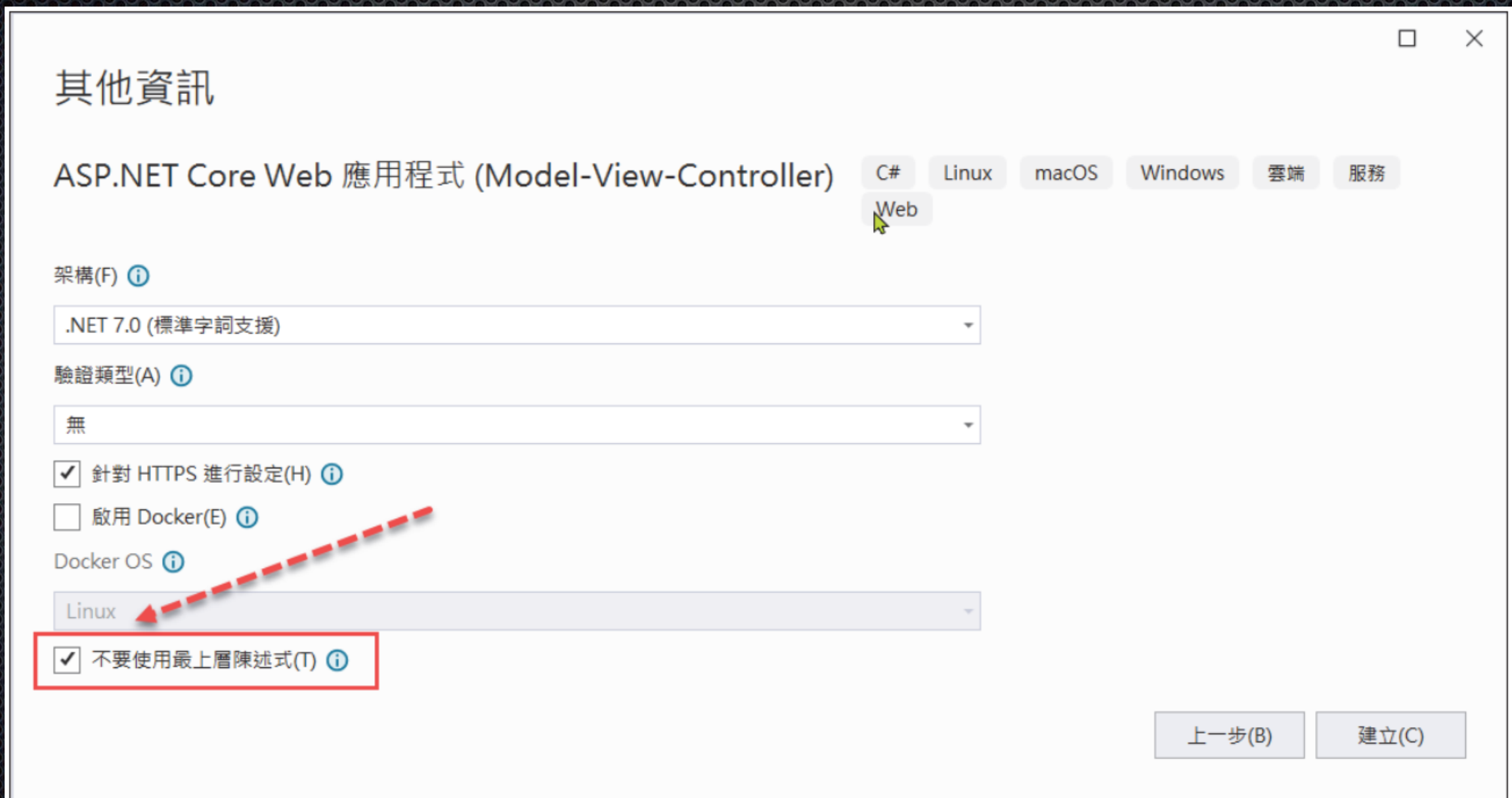
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

最上層陳述式

是否使用最上層陳述式

建立 ASP.NET Core MVC 專案時，預設會使用 Top Level Statement，這部分是在建立專案時【不要使用最上層陳述式】來決定，若勾選會產出 Main() 方法，反之則無



其他資訊

ASP.NET Core Web 應用程式 (Model-View-Controller) C# Linux macOS Windows 雲端 服務 Web

架構(F) ⓘ
.NET 7.0 (標準字詞支援)

驗證類型(A) ⓘ
無

☒ 針對 HTTPS 進行設定(H) ⓘ

☐ 啟用 Docker(E) ⓘ

Docker OS ⓘ
Linux

☒ 不要使用最上層陳述式(T) ⓘ

上一步(B) 建立(C)

Program.cs –建立 Host 主機

Program 的 Main()主要任務是建立 Host 主機：

- ① Host是裝載與執行.NET應用程式的主機環境，它封裝了所有App 資源，如Server、Middleware、DI和 Configuration
- ② Host封裝了前述種種相互依賴的服務，其目的只有一個，便是「生命週期管理」控制App應用程式啟動，及順利關閉Host主機

設定Host或App組態(1)

```
//自訂ContentRoot & WebRoot
//變更EnvironmentName, ContentRootPath, WebRootPath
var builder = WebApplication.CreateBuilder(new WebApplicationOptions
{
    Args = args,
    ApplicationName = typeof(Program).Assembly.FullName,
    EnvironmentName = Environments.Staging,
    ContentRootPath = Directory.GetCurrentDirectory(),
    WebRootPath =
        Path.Combine(Directory.GetCurrentDirectory(), "StaticFilesLibrary")
});

Console.WriteLine($"Application Name: {builder.Environment.ApplicationName}");
Console.WriteLine($"Environment Name: {builder.Environment.EnvironmentName}");
Console.WriteLine($"ContentRoot Path: {builder.Environment.ContentRootPath}");
Console.WriteLine($"WebRoot Path: {builder.Environment.WebRootPath}");
```


設定Host或App組態(2)

```
//加入Configuration組態設定
builder.Configuration.AddJsonFile("hostsettings.json", optional:true);
builder.Configuration.AddEnvironmentVariables(prefix: "PREFIX_");
builder.Configuration.AddCommandLine(args);

//設定組態檔完整路徑
string path = Path.Combine(Directory.GetCurrentDirectory(), "ConfigFiles");

//加入自訂組態檔
var config = builder.Configuration;
config.AddJsonFile(Path.Combine(path, "FutureCorp.json"), optional: true,
reloadOnChange: true); //載入自訂JSON組態檔
//載入自訂INI組態檔
config.AddIniFile(Path.Combine(path, "Mobile.ini"), true, true);
//載入自訂XML組態檔
config.AddXmlFile(Path.Combine(path, "Computer.xml"), true, true);
//載入自訂JSON組態檔
config.AddJsonFile(Path.Combine(path, "Device.json"), true, true);
string path2 = Path.Combine(Directory.GetCurrentDirectory(), "Configuration");
config.AddJsonFile(Path.Combine(path2, "Food.json"), true, true);
```


設定Host或App組態(3)

```
config.AddInMemoryCollection(new Dictionary<string, string>
{
    {"Asia:employees:1", "Mary"},
    {"Asia:employees:2", "John"},
    {"Asia:employees:3", "Kevin"},
    {"Asia:employees:4", "David"},
    {"Asia:employees:5", "Rose"}
});

//Add Logging Provider
var logging = builder.Logging;
logging.ClearProviders();
logging.AddConsole();
logging.AddDebug();
logging.AddEventSourceLogger();
logging.AddEventLog(); //for windows only
logging.AddTraceSource(new System.Diagnostics.SourceSwitch("loggingSwitch",
"Verbose"), new TextWriterTraceListener("LoggingService.txt"));
logging.AddAzureWebAppDiagnostics();
logging.AddApplicationInsights();
```


DI相依性注入與 Middleware 中介元件

4-3-4

Program.cs兩類重要設定

- ① Service 相依性註冊
- ② Middleware中介軟體設定

在DI Container容器註冊服務

DI容器註冊介於CreateBuilder()及Build()方法間

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllersWithViews();
//取得組態中資料庫連線設定
string connectionString =
builder.Configuration.GetConnectionString("DatabaseContext");

//註冊EF Core的DatabaseContext
builder.Services.AddDbContext<DatabaseContext>(options =>
options.UseSqlServer(connectionString));

//在DI Container中註冊DeveloperOptions類別
builder.Services.Configure<DeveloperOptions>(options =>
builder.Configuration.GetSection("Developer").Bind(options));

var app = builder.Build();
```


Middleware中介軟體設定

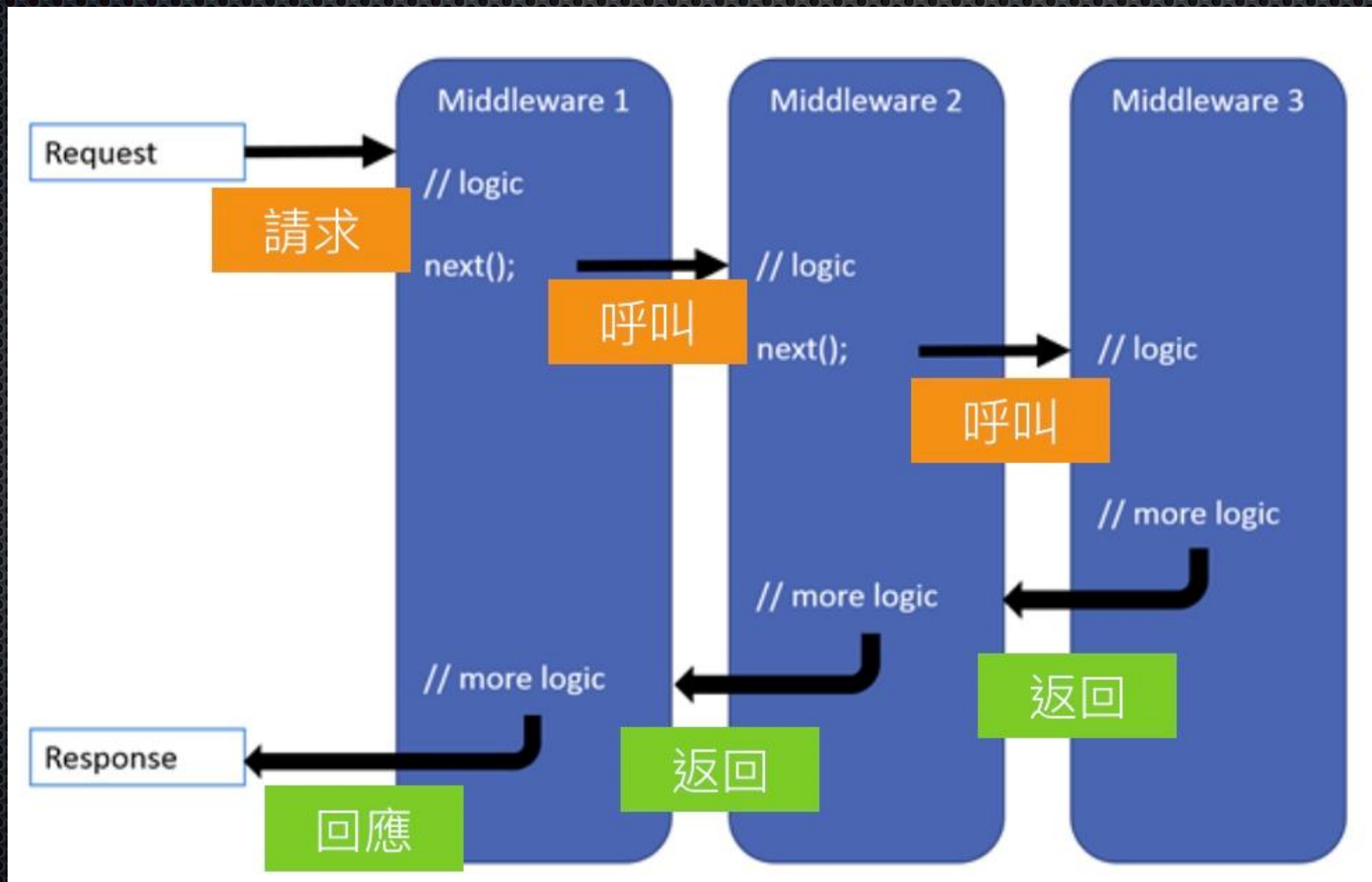
```
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error"); //一般例外頁
    app.UseHsts(); //HTTP Strict Transport Security Protocol
}

app.UseHttpsRedirection(); //將 HTTP 轉向 HTTPS
app.UseStaticFiles(); //靜態檔服務
app.UseCookiePolicy(); //Cookie Policy
app.UseRouting(); //路由
app.UseRequestLocalization(); //根據用戶端提供的資訊自動設定要求的文化特性資訊
app.UseCors(); //CORS 跨源資源分享
app.UseAuthentication(); //驗證
app.UseAuthorization(); //授權
app.UseSession(); //使用 Session
app.UseResponseCompression(); //回應壓縮
app.UseResponseCaching(); //回應快取

app.Run();
```


Middleware 元件運作方式與順序

HTTP 請求管線是由一系列 Middleware 元件組成，每個元件皆有不同功用，並以非同步方式在HttpContext 上作業，前一個元件會叫用下一個元件，或終止請求的執行

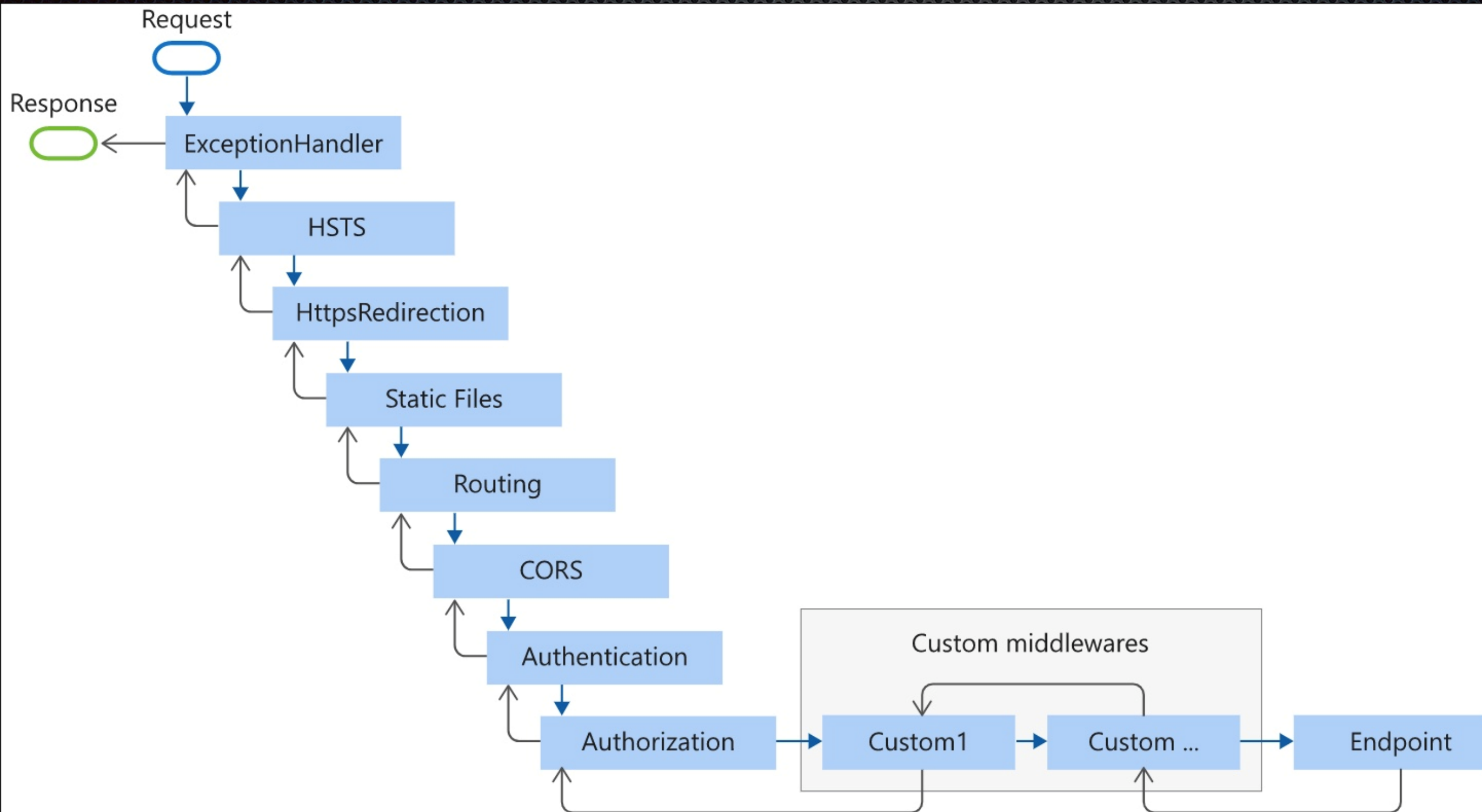


ASP.NET Core 內建的 Middleware 元件

Middleware 元件	說明	順序
Authentication	提供驗證支援	在需要 HttpContext.User 之前。OAuth 回呼的終端機
Authorization	提供授權支援	緊接在驗證中介軟體之後
Cookie Policy	追蹤使用者對用於儲存個人資訊的同意，並強制執行 Cookie 欄位的最低標準，例如 secure 和 SameSite	在發出 Cookie 的中介軟體之前。例如：驗證、工作階段、MVC (TempData)
CORS	設定跨原始來源 (Cross-Origin) 資源共用	在使用 CORS 的元件之前
Developer-ExceptionPage	產生包含錯誤資訊的頁面，此資訊僅供開發環境使用	在產生錯誤的元件之前。當環境為開發環境時，專案範本會自動將此中介軟體註冊為管線中的第一個中介軟體
Diagnostics	提供開發人員例外狀況頁面、例外狀況處理、狀態字碼頁，以及新應用程式的預設網頁的數個個別中介軟體	在產生錯誤的元件之前。終端機的例外狀況，或為新的應用程式提供預設的網頁
Forwarded Headers	將設為 Proxy 的標頭轉送到目前請求	在使用更新欄位元件前。例如：scheme、host、client IP

每個元件說明請參考書籍表4-1

Middleware元件執行順序



Configuration 組態

4-3-5

Configuration組態

- ASP.NET Core 組態是基於 Key-Value Pairs 形式，組態提供者（Configuration Providers）從各種組態來源讀取資料後，再以 Key-Value成對方式儲存在組態系統中
- ASP.NET Core 專案預設有 `launchSettings.json` 和 `appsettings.json` 組態檔，前者是本機開發電腦環境組態檔，後者是給應用程式使用的組態檔

appsettings.json

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.AspNetCore.HttpLogging.HttpLoggingMiddleware": "Information"
    }
  },
  "AllowedHosts": "*",
  "ConnectionStrings": {
    "DbContext": "Server=(localdb)\\mssqllocaldb;Database=ProductDB;Trust
  },
  "Developer": {
    "Name": "聖殿祭司",
    "Email": "dotnetcool@gmail.com",
    "Website": "https://www.codemagic.com.tw"
  }
}
```


以程式讀取Configuration組態

```
@using Microsoft.Extensions.Configuration
```

```
@inject IConfiguration configuration
```

```
<p>DatabaseContext資料庫連線 : @configuration["ConnectionStrings:DatabaseContext"]</p>
```

Developer資訊如下:

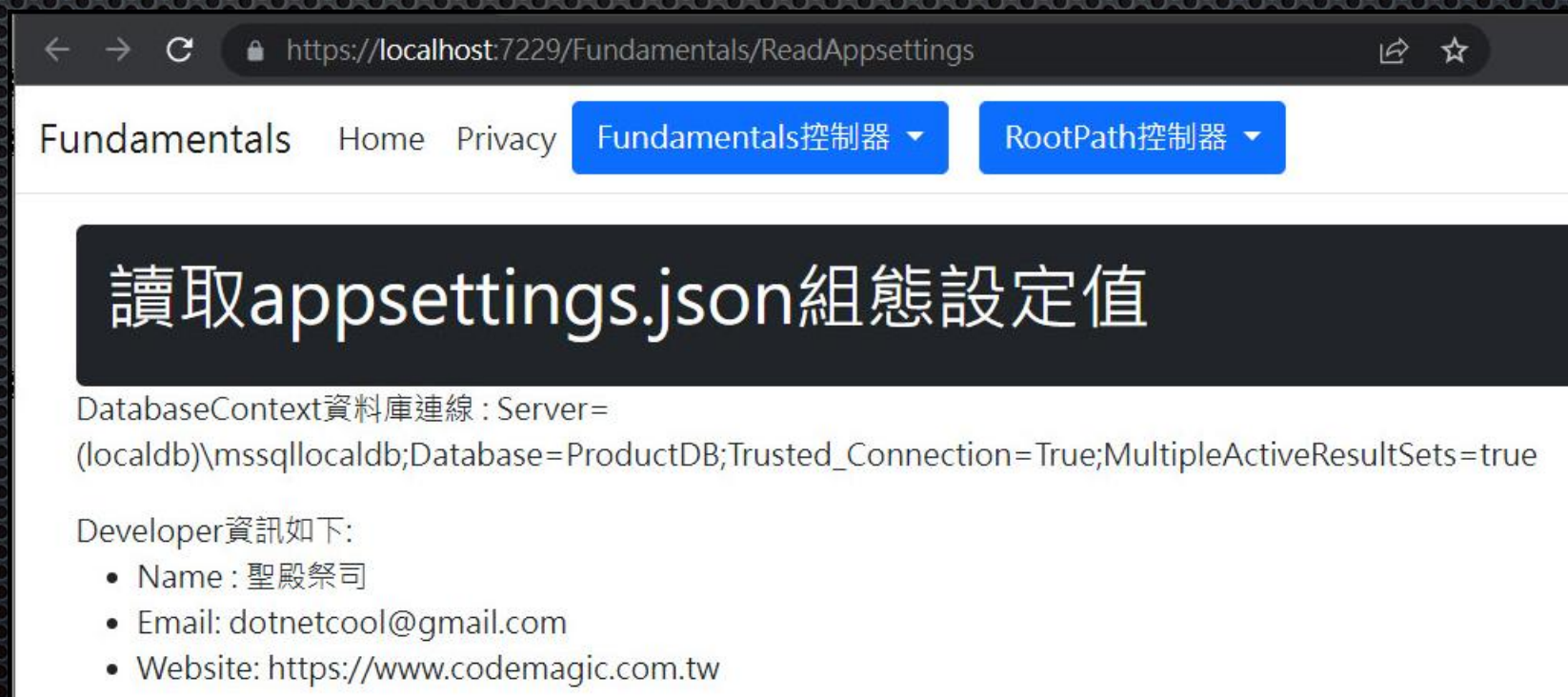
```
<ul>
```

```
<li>Name : @(configuration.GetValue<string>("Developer:Name"))</li>
```

```
<li>Email: @(configuration.GetValue<string>("Developer:Email", "找不到Email"))</li>
```

```
<li>Website: @(configuration.GetSection("Developer:Website").Value)</li>
```

```
</ul>
```



Options Pattern 選項模式

4-3-6

Options Pattern 選項模式

- 應用程式直接存取組態值會造成緊密耦合的問題
- ASP.NET Core 使用 Options Pattern 選項模式抽象化背後的組態系統，將組態資料先繫結到「類別」，應用程式再存取該類別，使程式不直接相依組態系統
- 日後組態資料結構有任何的調整，應用程式也不必連動做修改，唯一需要調整的是 Options 類別的設定

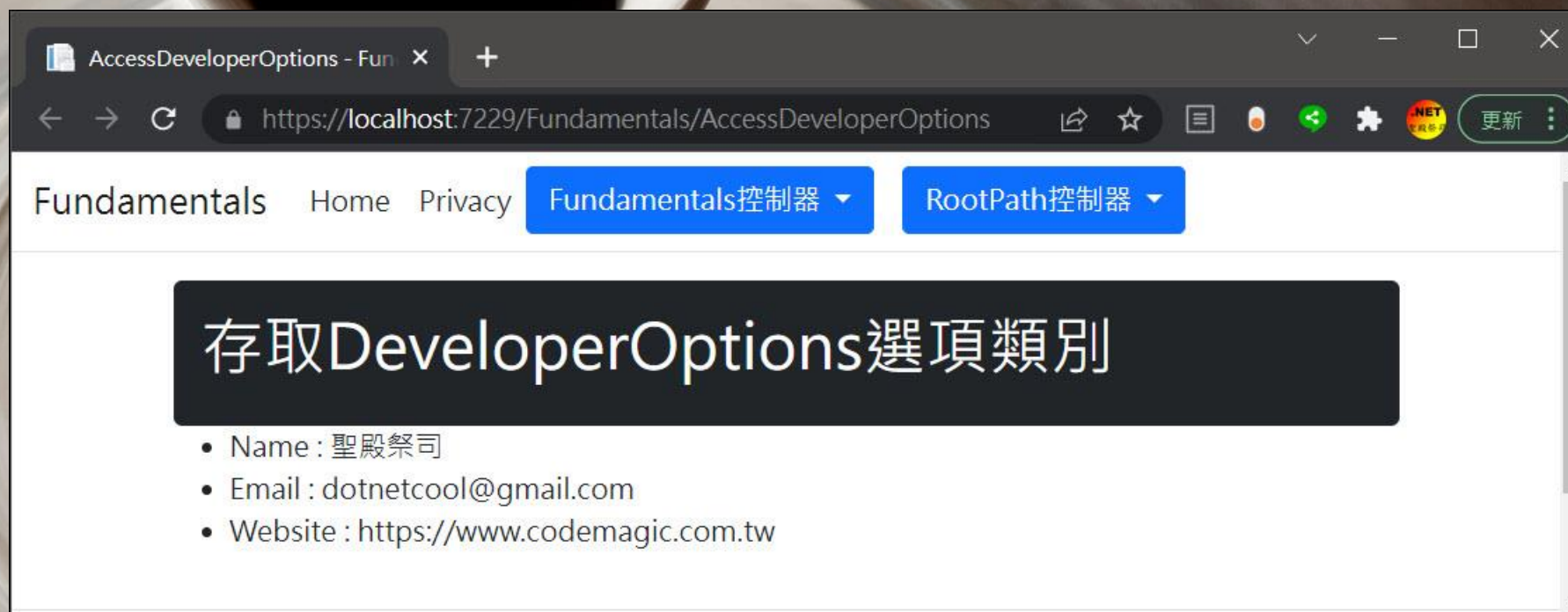
Options 類別兩個要求

- 必須為非抽象類別
- 建構函式必須為 public 且無參數

選項模式建立及使用四步驟

- ① 建立組態資料，並載入組態系統中
- ② 建立 Options 類別
- ③ 在 DI Container 中以 `builder.Services.Configure<T>` 方法註冊 Options 類別
- ④ 在 Controller / View / Services 中存取 Options 類別

範例 4-1 透過 Options 選項模式讀取組態設定



Environment 環境

4-3-7

Environment 環境

- ASP.NET Core 執行時會讀取 **ASPNETCORE_ENVIRONMENT** 環境變數
- 判斷它是 **Development**、**Staging** 或 **Production** 環境，依序代表**開發**、**預備**和**生產**環境，依環境的不同而載入對映程式或設定

```
"profiles": {  
  "http": {  
    "commandName": "Project",  
    "dotnetRunMessages": true,  
    "launchBrowser": true,  
    "applicationUrl": "http://localhost:5239",  
    "environmentVariables": {  
      "ASPNETCORE_ENVIRONMENT": "Development"  
    }  
  },  
  ...  
}
```

The diagram illustrates the configuration in the `Properties/launchSettings.json` file. A dashed box highlights the `"environmentVariables"` object, which contains the key `"ASPNETCORE_ENVIRONMENT"` with the value `"Development"`. An arrow points from this value to a box labeled "Development 開發環境". Another arrow points from the `"ASPNETCORE_ENVIRONMENT"` key to a box labeled "ASPNETCORE_ENVIRONMENT 環境變數".

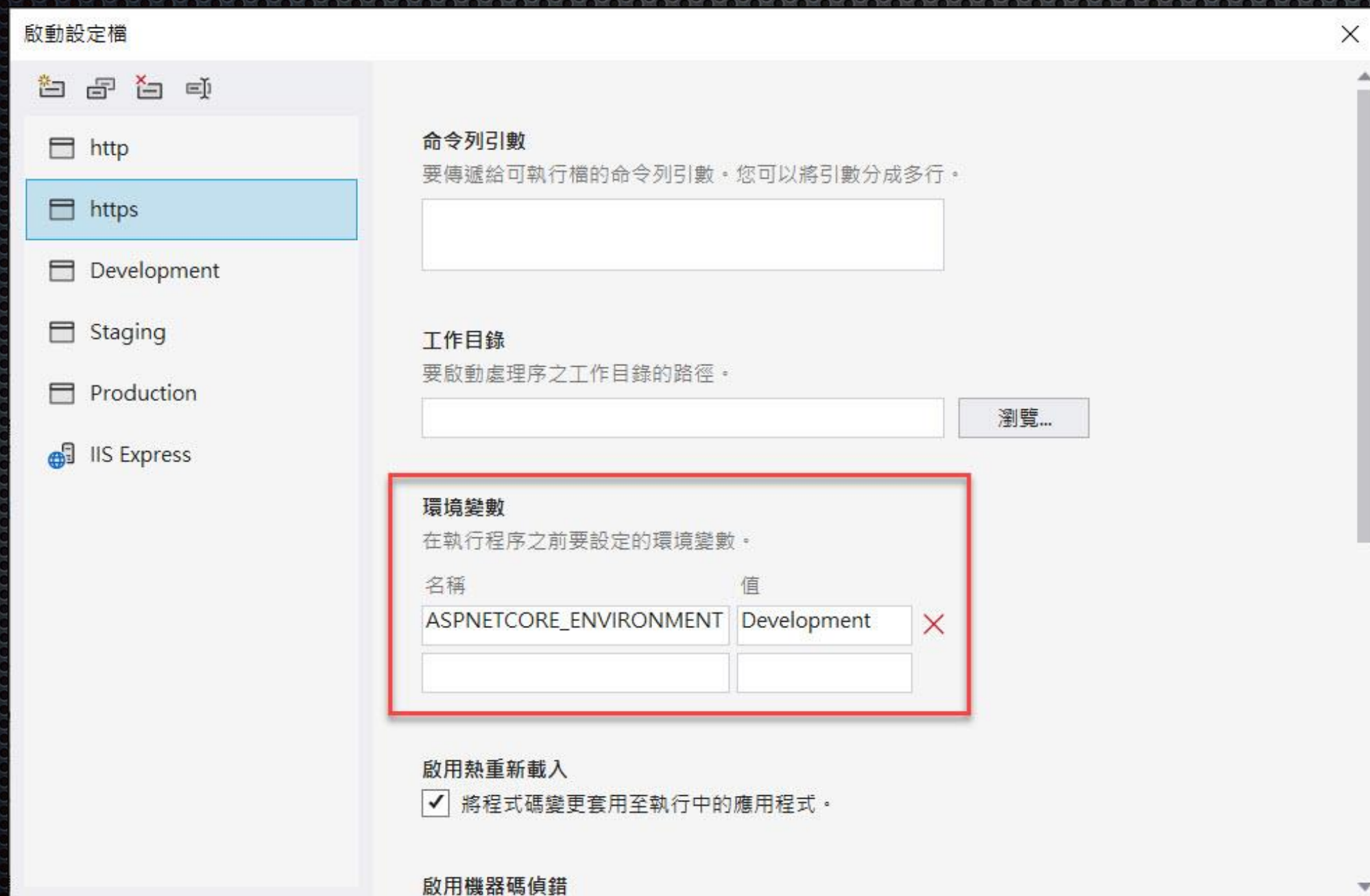
Properties/launchSettings.json

Properties/launchSettings.json

```
"Development": {
  "commandName": "Project",
  "dotnetRunMessages": true,
  "launchBrowser": true,
  "applicationUrl": "https://localhost:5000;http://localhost:5001",
  "environmentVariables": {
    "ASPNETCORE_ENVIRONMENT": "Development"
  }
},
"Staging": {
  "commandName": "Project",
  "dotnetRunMessages": true,
  "launchBrowser": true,
  "applicationUrl": "https://localhost:6500;http://localhost:6501",
  "environmentVariables": {
    "ASPNETCORE_ENVIRONMENT": "Staging"
  }
},
"Production": {
  "commandName": "Project",
  "dotnetRunMessages": true,
  "launchBrowser": true,
  "applicationUrl": "https://localhost:7000;http://localhost:7001",
  "environmentVariables": {
    "ASPNETCORE_ENVIRONMENT": "Production"
  }
},
```


在 VS 檢視專案的環境變數

專案的【屬性】→【偵錯】→【General】→【開啟 debug 啟動設定 UI】中檢視與編輯



ASP.NET Core依環境載入不同程式或設定之方式

- ① 在Program.cs判斷環境是否為Development、Staging和Production，進而決定使用不同的服務或Middleware元件
- ② 在 Controller 及 View 中使用環境變數
- ③ Environment 標籤協助程式

在Program.cs中依不同環境 載入不同的Middleware 元件

```
if (!app.Environment.IsDevelopment())  
{  
    app.UseExceptionHandler("/Home/Error"); //一般例外頁  
  
    app.UseHsts(); //HTTP Strict Transport Security Protocol  
}
```


在Controller及View使用環境變數

- 在 Controller 用 ViewData 傳遞給 View

```
public class FundamentalsController : Controller
{
    private readonly IWebHostEnvironment _env;
    public FundamentalsController(IWebHostEnvironment env)
    {
        _env = env;
    }

    //顯示環境名稱
    public IActionResult EnvironmentName()
    {
        ViewData["EnvName"] = _env.EnvironmentName;
        return View();
    }
}
```

↓ IWebHostEnvironment 相依性注入

↓ 讀取環境變數名稱

在View注入IWebHostEnvironment物件

```
@inject Microsoft.AspNetCore.Hosting.IWebHostEnvironment env
```

↑ IWebHostEnvironment 相依性注入

```
@{  
    //C# 8 switch expression  
    string DisplayEnvironment(string envName) =>  
        envName switch  
        {  
            "Development" => "開發環境",  
            "Staging" => "預備環境",  
            "Production" => "生產環境",  
            _ => "其他環境"  
        };  
}
```

↓ 將環境變數名稱傳遞給 local function

<p>目前環境是: @**DisplayEnvironment**(env.EnvironmentName)</p>

Environment 標籤協助程式

```
<environment include="Development">
  <h2><span class="badge badge-primary">Development 開發環境</span></h2>
</environment>

<environment include="Staging">
  <h2><span class="badge badge-danger">Staging 開發環境</span></h2>
</environment>

<environment include="Production">
  <h2><span class="badge badge-warning">Production 開發環境</span></h2>
</environment>

@section endJS
{
  <environment include="Development">
    <script>
      function Development() {
        alert("This is Development Environment.");
      }
    </script>
  </environment>
}
```


Content Root 與 Web Root

4-3-8

Content Root與Web Root

- Content Root：代表專案目前所在的基底路徑
- Web Root：專案對外公開靜態資產的目錄，預設路徑為{content root}/wwwroot

Content Root設定與讀取

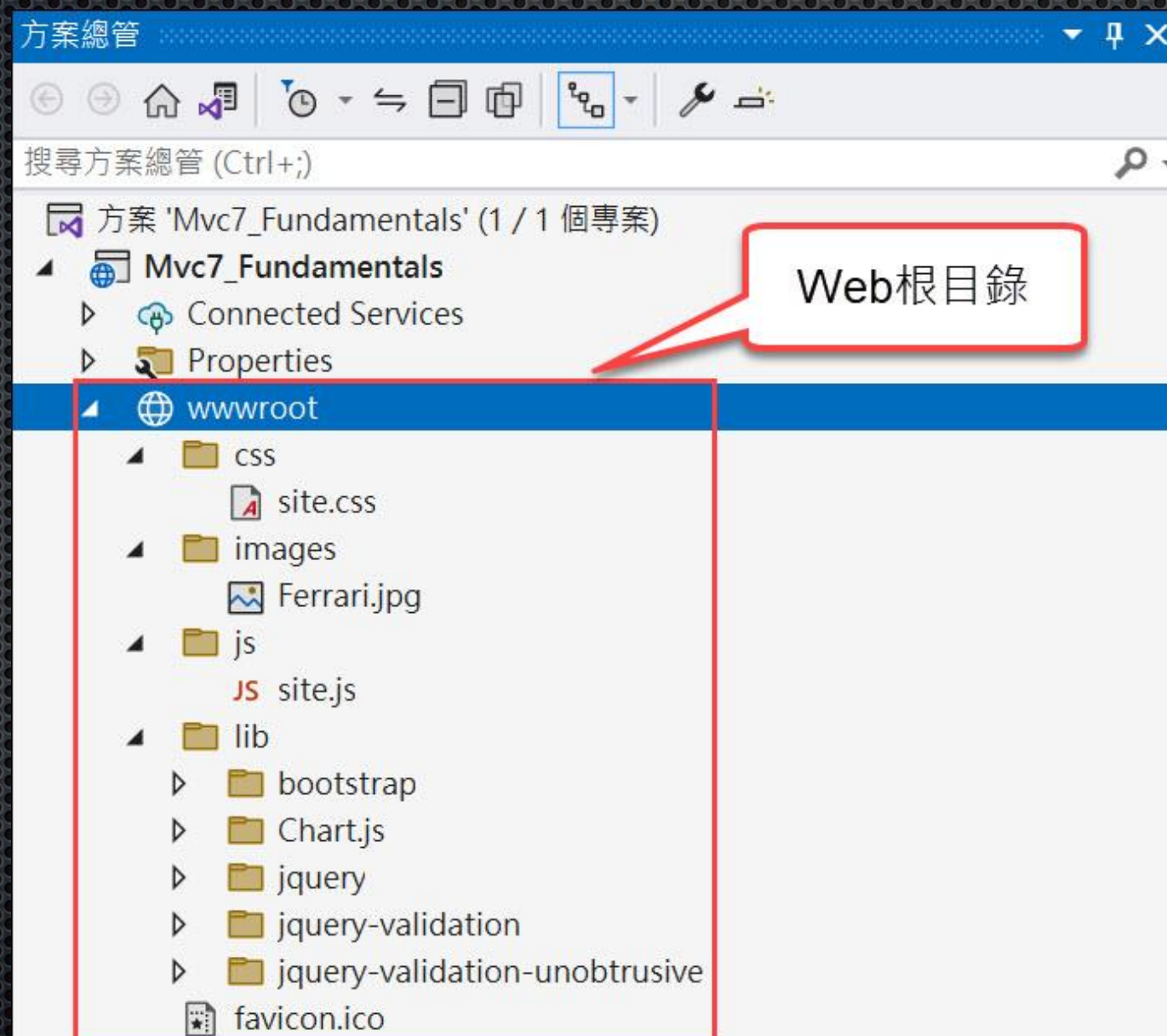
```
var builder = WebApplication.CreateBuilder(new WebApplicationOptions
{
    Args = args,
    ApplicationName = typeof(Program).Assembly.FullName,
    EnvironmentName = Environments.Staging,
    ContentRootPath = Directory.GetCurrentDirectory(),
    WebRootPath = Path.Combine(Directory.GetCurrentDirectory(), "StaticFilesLibrary")
});

Console.WriteLine($"Application Name: {builder.Environment.ApplicationName}");
Console.WriteLine($"Environment Name: {builder.Environment.EnvironmentName}");
Console.WriteLine($"ContentRoot Path: {builder.Environment.ContentRootPath}");
Console.WriteLine($"WebRoot Path: {builder.Environment.WebRootPath}");
```

Program.cs

Web Root設定與讀取

GetCurrentDirectory() 方法回傳的路徑再補上「**/wwwroot**」就是Web 根目錄路徑



在Controller 或 View 中讀取 Content Root 路徑

```
using Microsoft.AspNetCore.Hosting;

public class RootPathController : Controller
{
    private readonly IWebHostEnvironment _env;

    public RootPathController(IWebHostEnvironment env)
    {
        _env = env;
        string contentRoot = env.ContentRootPath;

        //Content Root Path
        public IActionResult ContentRootPath()
        {
            ViewData["ContentRootPath"] = _env.ContentRootPath;
            return View();
        }
    }
}
```

↑ IWebHostEnvironment 環境變數

↑ 讀取 ContentRootPath 屬性

↑ 讀取 ContentRootPath 屬性

Controllers/RootPathController.cs

用 Middleware 設定靜態檔目錄

```
app.UseStaticFiles(new StaticFileOptions
{
    FileProvider = new PhysicalFileProvider(Path.Combine(Directory.GetCurrentDirectory(),
        "StaticFilesLibrary"))
});

app.UseStaticFiles(new StaticFileOptions
{
    FileProvider = new PhysicalFileProvider(Path.Combine(Directory.GetCurrentDirectory(),
        "StaticFilesLibrary")),
    RequestPath = "/StaticFiles"
});
```

- 第一種沒有指定 RequestPath，請求路徑維持「~/...」
- 第二種指定了 RequestPath，View 中的的 src 請求路徑須改成「~/StaticFiles/...」

```

<br />

```


Logging 記錄

4-3-9

ASP.NET Core 內建的 Logging Provider 提供者

- Console
- Debug
- Windows 平台的 Event Tracing
- Windows 平台的事件記錄
- TraceSource
- Azure App Service (需參考
Microsoft.Extensions.Logging.AzureAppServices 的 NuGet 套件)
- Azure Application Insights (需參考
Microsoft.Extensions.Logging.ApplicationInsights 的 NuGet 套件)

Logging提供者輸出記錄方式

- Console提供者會輸出 Log 記錄到 Console 中
- 事件記錄提供者就寫入事件檢視器
- Azure Application Insights 儲存 Logs 在 Azure Application Insights 中
- 其他以此類推

預設 / 新增Logging提供者

```
var builder = WebApplication.CreateBuilder(args);
```

```
//加入 Logging Providers
```

```
builder.Logging.ClearProviders(); //清除所有 ILoggerProviders
```

```
builder.Logging.AddConsole();
```

```
builder.Logging.AddDebug();
```

```
builder.Logging.AddEventSourceLogger();
```

```
builder.Logging.AddEventLog(); //Windows Only
```

← 系統預設提供者

```
builder.Logging.AddAzureWebAppDiagnostics();
```

```
builder.Logging.AddApplicationInsights();
```

← 加入其他提供者

Program.cs

Log Level記錄層級

- 下表依重要性程度最高至最低

表 4-2 Log 記錄層級

等級	代碼	說明
None	6	不寫入 Log 訊息。指定 Logging 類別不應寫入任何的訊息。
Critical	5	記錄不可復原的應用程式，系統崩潰或災難性故障，需要立即關注。
Error	4	記錄當前執行流程因失敗而停止，而這個失敗是指當前的執行活動，但不是整個應用程式範圍的失敗。
Warning	3	記錄應用程式中的異常或非預事件外，但這些異常會不導致應用程式停止。
Information	2	用於記錄應用程式一般流程，而這些 Log 應該有一些長期價值。
Debug	1	在開發期間使用的互動式調查 Log 記錄。這些 Log 主要是包含 debugging 偵錯資訊，且沒有長期保存的價值。
Trace	0	此等級的 Log 包含大多數詳細資訊，這些訊息可能包含敏感的應用程式資料，這些訊息預設是 Disable 關閉，在 Production 生產環境中絕對不要開啟。

Log Level記錄層級作用

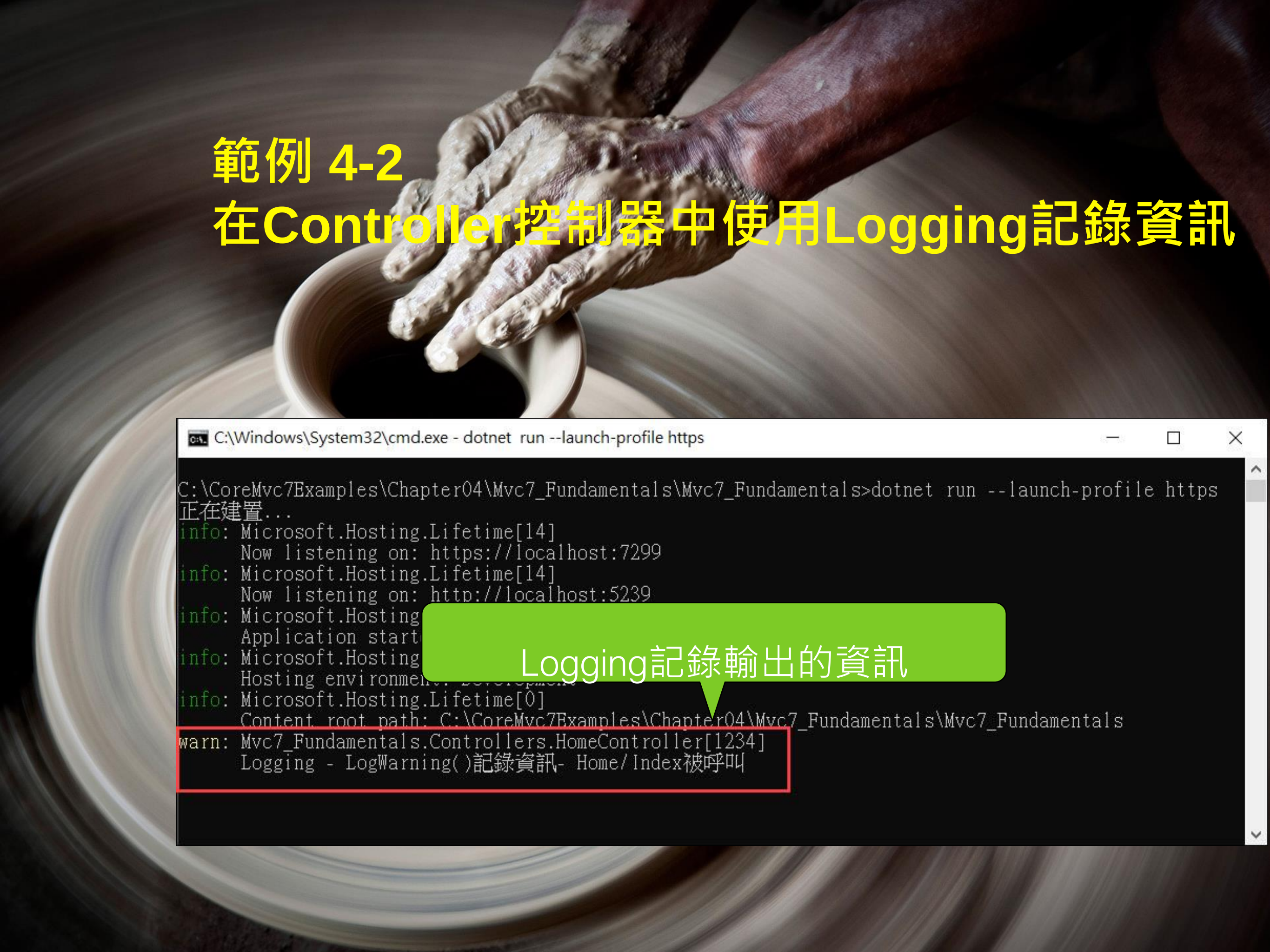
- Default 預設為「Information」，必須需使用至少跟它一樣或較高等級的方法，Log 才會寫入至記錄目的端
- 例如 LogDebug、LogInformation、LogWarning、LogError、LogCritical 方法皆可，但用LogDebug、LogTrace 方法就不做任何記錄

```
{  
  "Logging": {  
    "LogLevel": {  
      "Default": "Information",  
      "Microsoft": "Warning",  
      "Microsoft.Hosting.Lifetime": "Information"  
    }  
  }  
}
```

預設為Information層級

範例 4-2

在Controller控制器中使用Logging記錄資訊



```
C:\Windows\System32\cmd.exe - dotnet run --launch-profile https

C:\CoreMvc7Examples\Chapter04\Mvc7_Fundamentals\Mvc7_Fundamentals>dotnet run --launch-profile https
正在建置...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:7299
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5239
info: Microsoft.Hosting.Lifetime[14]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\CoreMvc7Examples\Chapter04\Mvc7_Fundamentals\Mvc7_Fundamentals
warn: Mvc7_Fundamentals.Controllers.HomeController[1234]
      Logging - LogWarning()記錄資訊- Home/Index被呼叫
```

Logging記錄輸出的資訊

The End

奚江華個人經歷簡介

- 著有ASP.NET 2.0、3.5、4.5、4.6等熱門暢銷書籍，以及**ASP.NET MVC 5.x範例完美演繹、ASP.NET Core 3.x、ASP.NET Core 7 MVC跨平台範例實戰演練**
- 歷任台灣微軟MSDN、TechEd、TechDay研討會講師
- 曾數屆當選微軟MVP
- 國內各大上市公司及大專院校之培訓講師
- 曾發表文章刊於ITHOME電腦週刊

碼魔法FB

- CodeMagic碼魔法軟體學院Facebook

www.facebook.com/codemagictw